INDEX

BEGINNER LEVEL

S. No.	Name of the Program	Bloom level with Cognitive process
1.	Write a program to perform various operations in the list: • Insertion • Deletion • Display	Apply (Using basic list operations)/ L3
2	Write a Program to sort the list using: • Bubble Sort • Quick Sort • Insertion sort • Merge Sort • Heap Sort Also, find the comparison on the basis of time complexity.	Apply (Sorting algorithm) / L3
3	Write a program to search the element using: • Linear Search • Binary Search Also, find the comparison on the basis of time complexity.	Apply (Basic search algorithm) / L3
4	Write a program to perform all operations: • For the Stack using an Array. • For the Queue using an Array. • For the Circular Queue using an Array. • Implement browser history using stacks.	Apply (Array manipulation, stack & queue operations) / L3
5	Write a program to evaluate the Postfix Notation.	Apply (Expression evaluation) / L3
6	Write a program to convert infix notation into Postfix Notation.	Apply (Expression conversion) / L3

INTERMEDIATE LEVEL

S.No	Name of the Program	Bloom level with Cognitive process
7	Write a program to perform the operations of the linked list: • Insertion • Deletion • Display	Apply (Linked list operation)/ L3
8	Write a program to perform the operations of the Circular linked list: • Insertion	Apply (Circular linked list) / L3

	• Deletion • Display	
9	Write a program to perform all operations: • For Stack using Linked List. • For Queue using Linked List • For Circular Queue using Linked List	Apply (Linked list manipulation)/ L3
10	Write a program to perform concatenation of two linked lists and Implement Polynomial Representation and Addition using Linked Lists	Apply (Linked list manipulation)/ L3
11	Write a program to perform the operations of the Double linked list: • Insertion • Deletion • Display	
12	Write a program to perform tree traversal methods.	Apply (Binary tree traversal) / L3
13	Write a program to perform insertion and deletion in the Binary Search Tree.	Apply (Binary Search Tree operations) / L3
14	Write a program to represent an undirected graph using the adjacency matrix to implement the graph and perform the operations with menu-driven options for the following tasks: 1. Create graph 2. Insert an edge 3. Print Adjacency Matrix 4. List all vertices that are adjacent to a specified vertex. 6. Exit program	Analyse (Graph operations, adjacency matrix) / L4
15	Simulation of Bloom Filter for Membership Testing	Apply L3

ADVANCED LEVEL

S.No.	Name of the Program	Bloom level with Cognitive process	Programs mapped with Ability Enhancement Courses
16	Kalnirnay: Conversion from Gregorian to Hindi calendar.	Analyse (Linked list manipulation) / L4	IKS
17	Design a system to optimize irrigation scheduling based on crop type, soil moisture, and weather.	Analyse (Linked list manipulation)/ L4	SDG
18	A queue-driven system to simulate the movement of planets across zodiac signs.	Analyse (Circular data structures)/ L4	IKS
19	A program that: -Uses efficient data structures to compute numerology -numbers for names. -Stores and organises results in a structured way.	Analyse (Linked list operation) / L4	IKS

	-Analyses frequency and patterns in numerology data.		
20	A program to classify plant or animal species based on environmental features (e.g., habitat, pollution tolerance).	Analyse (Linked list operation)/ L4	EVS
21	System to track and visualise personal health metrics (e.g., step count, calories burned, sleep).	Analyse (Circular linked list) / L4	IKS
22	Design a system that organises and preserves Vedic texts in digital form, ensuring easy access to verses, rituals, and philosophical content for future generations.	Create (Circular linked list) / L6	IKS
23	System that identifies and classifies common suffixes in Sanskrit words, which is essential for understanding word formation and grammar.	Analyse (Linked list operation) / L4	IKS
24	Case study on Improving Quality Education through Efficient Data Structures	L2/ L3/ L4/ L5/ L6	SDG
25	System that automates loan processing tasks, including loan eligibility checking, interest estimation, and repayment scheduling.	L2/ L3/ L4/ L5/ L6	FinTech

Practical 1:

Write a program to perform various operations in the list:

- Insertion
- Deletion
- Display

Solution:

```
lst=[]
while True:
    print("1.Insertion\n2.Deletion\n3.Display\n4.Exit")
    ch=int(input("Enter choice:"))
    if ch==1:
        x=int(input("Enter element:"))
        p=int(input("Enter position:"))
        lst.insert(p,x)
    elif ch==2:
        x=int(input("Enter element to delete:"))
        if x in lst:
            lst.remove(x)
        else:
            print("Element not found")
    elif ch==3:
        print(lst)
    elif ch==4:
        break
    else:
        print("Invalid choice")
```

```
1.Insertion
2.Deletion
3.Display
4.Exit
Enter choice:1
Enter element:10
Enter position:3
1.Insertion
2.Deletion
3.Display
4.Exit
Enter choice:3
[10]
1.Insertion
2.Deletion
3.Display
4.Exit
Enter choice:
```

Practical 2:

Write a Program to sort the list using:

- Bubble Sort
- Quick Sort
- Insertion sort
- Merge Sort
- Heap Sort

Also, find the comparison on the basis of time complexity

Solution:

```
def bubble_sort(a):
    a=a.copy()
    for i in range(len(a)):
        for j in range(len(a)-i-1):
            if a[j]>a[j+1]:
                a[j],a[j+1]=a[j+1],a[j]
    return a

def quick_sort(a):
    if len(a)<=1:
        return a
    p=a[len(a)//2]
    l=[x for x in a if x<p]
    m=[x for x in a if x==p]
    r=[x for x in a if x>p]
    return quick_sort(l)+m+quick_sort(r)

def insertion_sort(a):
    a=a.copy()
    for i in range(1,len(a)):
        key=a[i]
        j=i-1
        while j>=0 and a[j]>key:
            a[j+1]=a[j]
            j-=1
        a[j+1]=key
    return a

def merge_sort(a):
    if len(a)<=1:
        return a
    mid=len(a)//2
    l=merge_sort(a[:mid])
    r=merge_sort(a[mid:])
```

```
    result=[]
    i=j=0
    while i<len(l) and j<len(r):
        if l[i]<=r[j]:
            result.append(l[i])
            i+=1
        else:
            result.append(r[j])
            j+=1
    return result+l[i:]+r[j:]

def heap_sort(a):
    a=a.copy()
    n=len(a)
    def heapify(n,i):
        largest=i
        l=2*i+1
        r=2*i+2
        if l<n and a[l]>a[largest]:
            largest=l
        if r<n and a[r]>a[largest]:
            largest=r
        if largest!=i:
            a[i],a[largest]=a[largest],a[i]
            heapify(n,largest)
    for i in range(n//2-1,-1,-1):
        heapify(n,i)
    for i in range(n-1,0,-1):
        a[0],a[i]=a[i],a[0]
        heapify(i,0)
    return a

a=list(map(int,input("Enter list elements: ").split()))
print("Bubble Sort:",bubble_sort(a))
print("Quick Sort:",quick_sort(a))
print("Insertion Sort:",insertion_sort(a))
print("Merge Sort:",merge_sort(a))
print("Heap Sort:",heap_sort(a))
```

```
Enter list elements: 50 20 10 30 40
Bubble Sort: [10, 20, 30, 40, 50]
Quick Sort: [10, 20, 30, 40, 50]
Insertion Sort: [10, 20, 30, 40, 50]
Merge Sort: [10, 20, 30, 40, 50]
Heap Sort: [10, 20, 30, 40, 50]
```

Practical 3:
Write a program to search the element using:
- Linear Search
- Binary Search

Also, find the comparison on the basis of time complexity.

Solution:

```
def linear_search(a,x):
    for i in range(len(a)):
        if a[i]==x:
            return i
    return -1

def binary_search(a,x):
    low=0
    high=len(a)-1
    while low<=high:
        mid=(low+high)//2
        if a[mid]==x:
            return mid
        elif a[mid]<x:
            low=mid+1
        else:
            high=mid-1
    return -1

a=list(map(int,input("Enter list elements: ").split()))
x=int(input("Enter element to search: "))
print("Linear Search:",linear_search(a,x))
a.sort()
print("Sorted list:",a)
print("Binary Search:",binary_search(a,x))
```

```
Enter list elements: 20 50 30 10 40
Enter element to search: 20
Linear Search: 0
Sorted list: [10, 20, 30, 40, 50]
Binary Search: 1
```

Practical 4:

Write a program to perform all operations:

- For the Stack using an Array.
- For the Queue using an Array.
- For the Circular Queue using an Array.
- Implement browser history using stacks

Solution:

Stack Using an Array

```
stack=[]
n=int(input("Enter stack size: "))
while True:
    print("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Exit")
    ch=int(input("Enter choice: "))
    if ch==1:
        if len(stack)<n:
            stack.append(int(input("Enter element: ")))
        else:
            print("Stack Overflow")
    elif ch==2:
        if stack:
            print("Deleted:",stack.pop())
        else:
            print("Stack Underflow")
    elif ch==3:
        print(stack[-1] if stack else "Stack is empty")
    elif ch==4:
        print(stack)
    elif ch==5:
        break
```

```
Enter stack size: 3
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 1
Enter element: 10
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice: 3
10
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter choice:
```

Queue Using an Array

```
n=int(input("Enter queue size: "))
q=[None]*n
front=rear=-1
while True:
    print("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit")
    ch=int(input("Enter choice: "))
    if ch==1:
        if rear==n-1:
            print("Queue Overflow")
        else:
            x=int(input("Enter element: "))
```

```
        if front==-1:
            front=0
        rear+=1
        q[rear]=x
    elif ch==2:
        if front==-1 or front>rear:
            print("Queue Underflow")
        else:
            print("Deleted:",q[front])
            front+=1
            if front>rear:
                front=rear=-1
    elif ch==3:
        print(q[front:rear+1] if front!=-1 else [])
    elif ch==4:
        break
```

```
Enter queue size: 3
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 1
Enter element: 10
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 3
[10]
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 2
Deleted: 10
```

Circular Queue Using an Array

```
n=int(input("Enter queue size: "))
q=[None]*n
front=rear=-1
while True:
    print("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit")
    ch=int(input("Enter choice: "))
    if ch==1:
        if (rear+1)%n==front:
            print("Queue Overflow")
        else:
            x=int(input("Enter element: "))
            if front==-1:
                front=rear=0
            else:
                rear=(rear+1)%n
            q[rear]=x
    elif ch==2:
        if front==-1:
            print("Queue Underflow")
        else:
            print("Deleted:",q[front])
            if front==rear:
                front=rear=-1
            else:
                front=(front+1)%n
    elif ch==3:
```

```
Enter queue size: 3
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 1
Enter element: 10
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter choice: 3
10
```

```
        if front==-1:
            print([])
        else:
            i=front
            while True:
                print(q[i],end=" ")
                if i==rear:
                    break
                i=(i+1)%n
            print()
    elif ch==4:
        break
```

Browser History Using Stacks

```
back=[]
forward=[]
current="Home"
while True:
    print("\n1.Visit Page\n2.Back\n3.Forward\n4.Current Page\n5.Exit")
    ch=int(input("Enter choice: "))
    if ch==1:
        page=input("Enter URL: ")
        back.append(current)
        current=page
        forward.clear()
    elif ch==2:
        if back:
            forward.append(current)
            current=back.pop()
        else:
            print("No previous page")
    elif ch==3:
        if forward:
            back.append(current)
            current=forward.pop()
        else:
            print("No forward page")
    elif ch==4:
        print("Current Page:",current)
    elif ch==5:
        break
```

```
1.Visit Page
2.Back
3.Forward
4.Current Page
5.Exit
Enter choice: 4
Current Page: Home

1.Visit Page
2.Back
3.Forward
4.Current Page
5.Exit
Enter choice: 1
Enter URL: youtube
```

Practical 5:
Write a program to evaluate the Postfix Notation.

Solution:

```
stack=[]
exp=input("Enter postfix expression: ").split()
for x in exp:
    if x.lstrip('-').isdigit():
        stack.append(int(x))
    else:
        b=stack.pop()
        a=stack.pop()
        if x=='+':
            stack.append(a+b)
        elif x=='-':
            stack.append(a-b)
        elif x=='*':
            stack.append(a*b)
        elif x=='/':
            stack.append(a/b)
        elif x=='^':
            stack.append(a**b)
print("Result:",stack.pop())
```

```
Enter postfix expression: 5 3 + 2 *
Result: 16
```

Practical 6:
Write a program to convert infix notation into Postfix Notation.

Solution:

```
def infix_to_postfix(exp):
    stack=[]
    result=""
    prec={'+':1,'-':1,'*':2,'/':2,'%':2,'^':3}
    for x in exp:
        if x.isalnum():
            result+=x
        elif x=='(':
            stack.append(x)
        elif x==')':
            while stack and stack[-1]!='(':
                result+=stack.pop()
            stack.pop()
        else:
            while stack and stack[-1]!='(' and (prec[stack[-1]]>prec[x] or (prec[stack[-1]]==prec[x]
and x!='^')):
                result+=stack.pop()
            stack.append(x)
    while stack:
        result+=stack.pop()
    return result

exp=input("Enter infix expression: ")
print("Postfix expression:",infix_to_postfix(exp.replace(" ","")))
```

```
Enter infix expression: (A+B)*C
Postfix expression: AB+C*
```